33

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

테스트 모듈과 프로바이더 오버라이드

제2부 NestJS 핵심 · 8장 테스트 기초

서버와 같은 조립, 필요한 곳만 대역



© 한상훈 NestJS 백엔드 마스터

생성자가 바뀌자 깨진 테스트

```
// 결제사 하나만 받던 시절에 쓴 테스트
const service = new ReservationsService(fakeGateway);
```

```
// 지금 생성자
  constructor(
    private readonly gateway: PaymentGateway,
    private readonly logger: TaggedLogger,
    private readonly notifications: NotificationsService,
  ) {
    this.logger.setTag('예매');
  }
```

○ ○ ○ 터미널

```
  × 결제사를 실제로 부르지 않고 예매를 시험한다 2ms
TypeError: Cannot read properties of undefined (reading 'setTag')
 ❯ new ReservationsService apps/ticketing/src/reservations/reservations.service.ts:24:17
```

© 한상훈 NestJS 백엔드 마스터

타입 오류는 어디에

터미널

```
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
(출력 없음, exit 0)     ← exclude에 **/*spec.ts

$ npm run test:types      ← tsc -p tsconfig.spec.json
reservations.service.spec.ts(9,19): error TS2554: Expected 3 arguments, but got 1.
reservations.service.spec.ts(11,24): error TS2554: Expected 3-5 arguments, but got 2.
```

```jsonc
// tsconfig.spec.json
{
  "extends": "./tsconfig.json",
  "compilerOptions": { "noEmit": true, "incremental": false },
  "files": [],
  "include": ["apps/*/src/**/*.ts", "libs/**/*.ts"]
}
```

→ vitest: 타입을 지우고 실행, 타입 검사 없음

→ 서버 빌드 설정: 테스트 파일 제외

→ 커밋 전: `npm test`와 `npm run test:types` 둘 다

© 한상훈 NestJS 백엔드 마스터

리허설 무대의 대역 배우



→ 주연 대신 대역, 조명과 동선은 본공연 그대로

→ 테스트 모듈: 서버와 같은 방식의 조립

→ 오버라이드: 진짜로 부르면 안 되는 배역만 대역으로

© 한상훈 NestJS 백엔드 마스터

Test.createTestingModule

→ @nestjs/testing의 함수: 서버와 같은 DI 컨테이너를 테스트 안에 생성

→ 인자: @Module과 같은 칸 (providers, imports, controllers)

→ compile(): 인스턴스 생성, 생성자 인자 연결

→ get(): 만들어진 인스턴스 꺼내기

→ 생성자 인자는 Nest가 채움: 테스트에 new 없음

→ 예: 첫 화면 문구 테스트, controllers에 AppController, getHello 직접 호출

© 한상훈 NestJS 백엔드 마스터

빠진 프로바이더

```
it('테스트 모듈로 예매 서비스를 만든다', async () => {
  const moduleRef = await Test.createTestingModule({
    providers: [
      ReservationsService,
      { provide: PaymentGateway, useValue: { charge: vi.fn() } },
    ],
  }).compile();

  expect(moduleRef.get(ReservationsService)).toBeDefined();
});
```

○ ○ ○ 터미널

```
Error: Nest can't resolve dependencies of the ReservationsService (PaymentGateway, ?, NotificationsService).
Please make sure that the argument TaggedLogger at index [1] is available in the RootTestModule module.
```

→ 빠진 것의 이름(TaggedLogger)과 위치(index [1])

→ 실패 시점: 생성자 실행 전, compile 단계

© 한상훈 NestJS 백엔드 마스터

목록에 가짜를 적기

```
  const sendConfirmed = vi.fn().mockReturnValue('문자 발송');
  const moduleRef = await Test.createTestingModule({
    providers: [
      ReservationsService,
      TaggedLogger,
      { provide: PaymentGateway, useValue: { charge: async (amount: number) => ({ approvalId: 'stub-1', amount }) } },
      { provide: NotificationsService, useValue: { sendConfirmed } },
    ],
  }).compile();

  const result = await moduleRef.get(ReservationsService).reserve('A-3-15', 110000, '김민수');
```

○ ○ ○ 터미널

```
TaggedLogger #1 생성
[예매] A-3-15 결제 완료 (unknown)
 ✓ 목록에 가짜를 적어서 예매 서비스를 만든다 314ms
```

→ 로거: 진짜, 결제사·알림: 가짜 (useValue)

→ 목록이 길어지는 문제: 알림 서비스 안의 Discovery, 문자·메일·푸시 발송기, 결제 설정 토큰…

© 한상훈 NestJS 백엔드 마스터

앱 모듈을 통째로

```
it('앱 모듈을 통째로 가져와 예매한다', async () => {
  const moduleRef = await Test.createTestingModule({ imports: [AppModule] }).compile();
  const service = moduleRef.get(ReservationsService);

  const result = await service.reserve('A-3-15', 110000, '김민수');
  expect(result.approvalId).toBe('fake-110000');
});
```

○ ○ ○ 터미널

```
결제 설정: {"mode":"http","timeoutMs":3000}
   × 앱 모듈을 통째로 가져와 예매한다 13ms
TypeError: Failed to parse URL from undefined/token
```

→ .env를 읽는 곳: main.ts의 process.loadEnvFile(), 테스트는 main.ts를 안 거침

→ 결제 모듈 팩토리: compile 중에 결제사로 토큰 요청

© 한상훈 NestJS 백엔드 마스터

세 가지 결과

환경	결과	출력
PG_URL 없음	compile 실패	`Failed to parse URL from undefined/token`
결제사 꺼짐	compile 실패	`fetch failed, connect ECONNREFUSED ::1:5032`
결제사 켜짐	진짜 결제	`expected 'pg-110000' to be 'fake-110000'`

→ 셋째 줄: 테스트를 돌릴 때마다 결제사에 실제 결제 요청

© 한상훈 NestJS 백엔드 마스터

overrideProvider

```
it('결제사만 페이크로 바꾸고 앱 모듈을 통째로 가져온다', async () => {
  const moduleRef = await Test.createTestingModule({ imports: [AppModule] })
    .overrideProvider(PaymentGateway)
    .useClass(FakePaymentGateway)
    .compile();
  const service = moduleRef.get(ReservationsService);

  const result = await service.reserve('A-3-15', 110000, '김민수');
  expect(result.approvalId).toBe('fake-110000');
  expect(moduleRef.get('LEGACY_PAYMENT')).toBeInstanceOf(FakePaymentGateway);
});
```

→ 앱 모듈은 그대로, PaymentGateway 토큰만 교체

© 한상훈 NestJS 백엔드 마스터

대역이 선 무대

터미널

```
FakePaymentGateway #1 생성
ConcertsService 생성
TaggedLogger #1 생성
TaggedLogger #2 생성
SeatsService 생성
[예매] A-3-15 결제 완료 (unknown)
  ✓ 결제사만 페이크로 바꾸고 앱 모듈을 통째로 가져온다 146ms
```

→ '결제 설정' 로그 없음: 원래 팩토리를 아예 안 부름, 토큰 요청 없음

→ 나머지는 서버와 같은 순서로 생성

→ `'LEGACY_PAYMENT'`(`useExisting` 별칭): 바뀐 페이크를 그대로 가리킴

© 한상훈 NestJS 백엔드 마스터

바꾸는 방법

→ `.overrideProvider(토큰).useClass(클래스)`: 클래스로 교체

→ `.overrideProvider(토큰).useValue(객체)`: 목이나 스텁 객체로 교체

→ `.overrideProvider(토큰).useFactory({ factory, inject })`: 함수로 교체

→ `.overrideModule(모듈).useModule(다른 모듈)`: 모듈 통째로 교체

→ 바꿀 대상: 결제사, 문자 발송처럼 밖으로 나가는 것

→ 결제 모듈: 앱 모듈에서 `forRootAsync`로 한 번 등록, 교체는 테스트 쪽 오버라이드로 (서버 코드 변경 없음)

© 한상훈 NestJS 백엔드 마스터

compile과 init

```
  const moduleRef = await Test.createTestingModule({ imports: [AppModule] })
    .overrideProvider(PaymentGateway)
    .useClass(FakePaymentGateway)
    .compile();
  const before = await service.reserve('A-3-15', 110000, '김민수');
  console.log('compile 뒤:', JSON.stringify(before));

  await moduleRef.init();
```

→ compile 뒤: 결제 성공, 문자 채널 없음 (채널을 모으는 곳: `NotificationsService.onModuleInit`)

→ init: 문자 발송함(`SmsOutbox`)이 발송사(5029)에 발신번호 요청 → 실패

○ ○ ○ 터미널

```
compile 뒤: {"id":1,"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"message":"지원하지 않는 채널: sms"}
 × 결제사만 페이크로 바꾸고 앱 모듈을 통째로 가져온다 178ms
   → fetch failed
 ❯ SmsOutbox.onModuleInit apps/ticketing/src/notifications/sms-outbox.ts:20:17
     20|     const res = await fetch(`${this.baseUrl}/sender`);
```

© 한상훈 NestJS 백엔드 마스터

문자 발송함도 대역으로

```
  const outbox = { enqueue: vi.fn() };
  const moduleRef = await Test.createTestingModule({ imports: [AppModule] })
    .overrideProvider(PaymentGateway)
    .useClass(FakePaymentGateway)
    .overrideProvider(SmsOutbox)
    .useValue(outbox)
    .compile();
```

→ useValue로 넣은 객체: onModuleInit이 없어서 발신번호 요청도 없음

→ 발송함 목: 실제로 보낼 문자를 기록

○ ○ ○ 터미널

```
compile 뒤: {"id":1,"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"message":"지원하지 않는 채널: sms"}
알림 채널: email, push, sms
init 뒤: {"id":2,"seatId":"A-3-16","approvalId":"fake-110000","amount":110000,"message":"[문자 → 이지은] A-3-16 예매가 확정되었습니다"}
발송함: [["이지은","A-3-16 예매가 확정되었습니다"]]
```

© 한상훈 NestJS 백엔드 마스터

수명주기와 스코프

→ `compile()`: 인스턴스 생성까지

→ `init()`: `onModuleInit`, `onApplicationBootstrap` 실행

→ `close()`: `onModuleDestroy` 등 종료 훅 실행, afterEach에서 호출

→ TRANSIENT 프로바이더와 `get()`:

```
TaggedLogger is marked as a scoped provider. Request and transient-scoped providers can't be used in combination with "get()" method. Please, use "resolve()" instead.
```

→ 요청마다·주입마다 새로 만드는 프로바이더: `await moduleRef.resolve(TaggedLogger)`

→ init을 빼먹은 테스트: 운영과 다른 서버를 시험, 기댓값까지 틀리게 적을 위험

© 한상훈 NestJS 백엔드 마스터

같이 돌리면 409

```
describe('SeatsModule', () => {
  let moduleRef: TestingModule;
  let seats: SeatsService;

  beforeAll(async () => {
    moduleRef = await Test.createTestingModule({ imports: [SeatsModule] })
      .overrideProvider(PerformancesService)
      .useValue({ findOne: (id: number) => ({ id, concertId: 1, startsAt: '2026-12-24T19:00' }) })
      .compile();
    seats = moduleRef.get(SeatsService);
    seats.open(1);
  });

  it('온라인에서 A-3-15를 붙잡는다', () => {
    expect(seats.hold(1, 'A-3-15').status).toBe('HELD');
  });

  it('현장 창구에서 A-3-15를 판다', () => {
    expect(seats.sell(1, 'A-3-15').status).toBe('SOLD');
  });
});
```

○ ○ ○ 터미널

```
$ npx vitest run
  apps/ticketing/src/seats/seats.module
 ✓ SeatsModule > 온라인에서 A-3-15를 붙잡는다 1ms
 × SeatsModule > 현장 창구에서 A-3-15를 판다 2ms

$ npx vitest run
  apps/ticketing/src/seats/seats.module -t
  "현장 창구"
 ✓ SeatsModule > 현장 창구에서 A-3-15를 판다 1ms
      Tests  1 passed | 1 skipped (2)
```

→ 실패 메시지: 'A구역 3열 15번은 이미 팔렸거나 선택된 좌석입니다'

→ `-t`로 하나만 실행: 온라인 붙잡기 테스트는 건너뜀 (skipped), 창구 판매는 통과

© 한상훈 NestJS 백엔드 마스터

테스트마다 새 모듈

```
beforeEach(async () => {
  moduleRef = await Test.createTestingModule({ imports: [SeatsModule] })
    .overrideProvider(PerformancesService)
    .useValue({ findOne: (id: number) => ({ id, concertId: 1, startsAt: '2026-12-24T19:00' }) })
    .compile();
  seats = moduleRef.get(SeatsService);
  seats.open(1);
});
```

○ ○ ○ 터미널

```
✓ SeatsModule > 온라인에서 A-3-15를 붙잡는다 135ms
✓ SeatsModule > 현장 창구에서 A-3-15를 판다 2ms
```

→ 싱글톤 필드(좌석표): 같은 테스트 모듈을 쓰는 모든 테스트가 공유

→ `beforeAll`: 빠르지만 테스트 순서에 따라 결과가 달라짐

→ 혼자 통과, 같이 실패: 앞 테스트가 남긴 상태부터 확인

→ 무거운 것만 `beforeAll` (예: DB 컨테이너), 데이터는 테스트마다 정리

© 한상훈 NestJS 백엔드 마스터

세 가지 방법

방법	쓰는 곳	조심할 것
new + 더블	서비스 하나의 규칙	생성자가 바뀌면 고칠 곳 많음, `test:types`로 확인
테스트 모듈 + `providers` 목록	서비스 몇 개의 연결	목록이 앱 모듈과 따로 놂
테스트 모듈 + `imports` + 오버라이드	모듈 구성까지 서버와 같게	부팅 때 하는 일이 따라옴, init과 close

→ 오버라이드 대상: 밖으로 나가는 것 (결제사, 문자 발송함, 외부 API), 초기화 훅 안의 호출 포함

→ 반복되는 조립 코드: 테스트용 조립 함수 하나로 (새 외부 호출이 생기면 한 곳에 추가)

© 한상훈 NestJS 백엔드 마스터

전체 실행

```
// reservations/reservations.module.spec.ts
  beforeEach(async () => {
    vi.resetAllMocks();
    moduleRef = await Test.createTestingModule({ imports: [AppModule] })
      .overrideProvider(PaymentGateway)
      .useClass(FakePaymentGateway)
      .overrideProvider(SmsOutbox)
      .useValue(outbox)
      .compile();
    await moduleRef.init();
  });
```

```
  afterEach(async () => {
    await moduleRef.close();
  });
```

→ npm run test:types: 에러 없음

○ ○ ○ 터미널

```
$ npx vitest run
 ✓ seats.module.spec.ts > SeatsModule > 온라인에서 A-3-15를 붙잡는다 318ms
 ✓ seats.module.spec.ts > SeatsModule > 현장 창구에서 A-3-15를 판다 3ms
 ✓ reservations.module.spec.ts > 예매 모듈 조립 > 결제사와 문자 발송함만 바꾸고 예매하면 확정 문자가 발송함에 들어간다 155ms
 ✓ reservations.module.spec.ts > 예매 모듈 조립 > 환불 쪽 별칭도 같은 페이크를 받는다 4ms
 (… 유닛 테스트 13개)
 Test Files  10 passed (10)
      Tests  17 passed (17)
```

데이터 블록: 전체 실행 10 files, 17 passed, Duration 1.92s(강사 PC 실측 2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

테스트 모듈의 규칙

"테스트 모듈은 서버와 같은 방식으로 조립하고,
진짜로 부르면 안 되는 것만 오버라이드로 바꿉니다."